Unit-3: Unsupervised Learning

Overview

Unsupervised Learning is a type of machine learning where the model is trained on unlabelled data, meaning there are no predefined outputs or target values. The goal of unsupervised learning is to find hidden patterns, structures, or relationships within the data.
Unlike supervised learning, where the model learns from input–output pairs, unsupervised learning tries to understand the underlying distribution or group similar data points together based only on their features.

Two of the most common approaches in unsupervised learning are:
Clustering: Grouping similar data points into clusters based on similarity or distance measures.

Example: grouping customers by buying behaviour or segmenting images into regions.

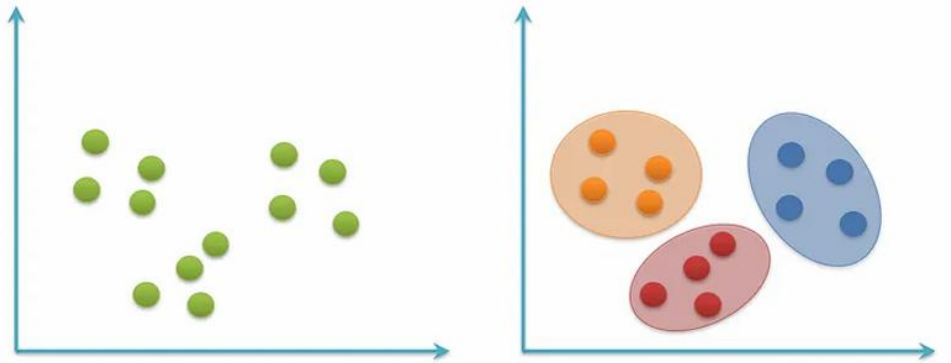
Association Rule Mining: Discovering interesting relationships or associations between variables in large datasets.

Example: finding which products are frequently bought together in market basket analysis.

Unsupervised learning techniques are widely used in data analysis, recommendation systems, pattern recognition, anomaly detection, and image processing, helping systems learn from data without explicit guidance.

Clustering:

- Clustering is an unsupervised machine learning technique, which partitions the entire dataset into a fixed number of clusters or groups in such a way that observations in each group exhibit similar behavior or characteristics and are homogeneous.
- The observations in the same group are more similar than the observations in other groups.
- It does it by finding some similar patterns in the unlabeled dataset such as shape, size, color, behavior, etc., and divides them as per the presence and absence of those similar patterns.



- It is an unsupervised learning method; hence no supervision is provided to the algorithm, and it deals with the unlabeled dataset.
- After applying this clustering technique, each cluster or group is provided with a cluster-ID.

- ML systems can use this id to simplify the processing of large and complex datasets.
- The clustering technique can be widely used in various tasks.
- Some most common uses of this technique are:
    - Market Segmentation
    - Statistical data analysis
    - Social network analysis
    - Image segmentation
    - Anomaly detection

Clustering algorithms

K-Means Clustering

- K-means clustering is a popular unsupervised machine learning algorithm used to group data points into K clusters based on their similarity.
- The K in K-means represents the number of clusters that the algorithm will create.

How does it work?

- Randomly choose the 'K' Centroids (for K Clusters).
- Compute the Euclidean distance of each observation with these Centroids.
- Assign the objects to clusters with shortest distance.
- Compute the new centroid (mean) of each cluster based on the observations assigned to each cluster.
- The K number of means obtained will become the new centroids for each cluster.
- Repeat step 2 to 4 till there is convergence. i.e. there is no movement of the observed points from one cluster to another, or threshold number of iterations have occurred.

Advantages

- Relatively simple to implement
- Can be used to scale up large datasets
- Can be used to form clusters of any shape and size

Limitations

- Sensitive to initial selection of centroids
- Sensitive to outliers in the data
- Cannot be used for non-linear data
- Cannot be used for categorical data

```python
#K-means Clustering
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs #make_blobs creates dataset useful for clustering
import matplotlib.pyplot as plt

# Step 1: Create sample data
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=1) #n_samples=300 means, total number of records to generate
                                                                              #centers=4 means, create 4 cluster centers
                                                                              #cluster_std=0.6 means, controls how spread out each cluster is
                                                                                 #(smaller means tighter clusters).
                                                                              #random_state=0 means, fixes the random seed so results are reproducible

# Step 2: Apply K-Means
kmeans = KMeans(n_clusters=4) #4 clusters will be formed
kmeans.fit(X) #trains the K-Means model on the data X
y_kmeans = kmeans.predict(X)

# Step 3: Visualize clusters
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis') #X[:, 0] is the first feature (x-axis), X[:, 1] is the second (y-axis)
                                                                #c=y_kmeans as cluster  size is 4, colors each point according to its
                                                                    #predicted cluster
                                                                #s=50 is the size of the marker
                                                                #cmap='viridis' maps those numeric values to distinct colors using the
                                                                    #viridis color palette
centers = kmeans.cluster_centers_ #stores the coordinates of each cluster's centroid
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='X')#centers[:,0] and centers[:,1] are centroid coordinates
                                                                                 #c='red' colors the centroid markers red.
                                                                                 #s=200 makes them much larger than data points so they stand out
                                                                                 #alpha=0.75 makes them slightly transparent
                                                                                 #marker='X' uses an X symbol to differentiate centroids
                                                                                    #from data points
plt.title("K-Means Clustering")
plt.show()
```

Compressing Image Using Vector Quantization (VQ)

- Vector Quantization (VQ) is a technique originally developed for data compression and later used in image processing and pattern recognition.
- In the context of images, it is a lossy compression method that reduces the number of colors (or pixel values) by grouping similar pixel values together.
- Every image consists of many pixels, each represented by 3 values (R, G, B).
- Many pixels have similar colour values.
- Instead of storing every pixel's unique RGB value, we can store only a few representative colours and map each pixel to the nearest representative.
- This reduces the amount of data needed to represent the image.

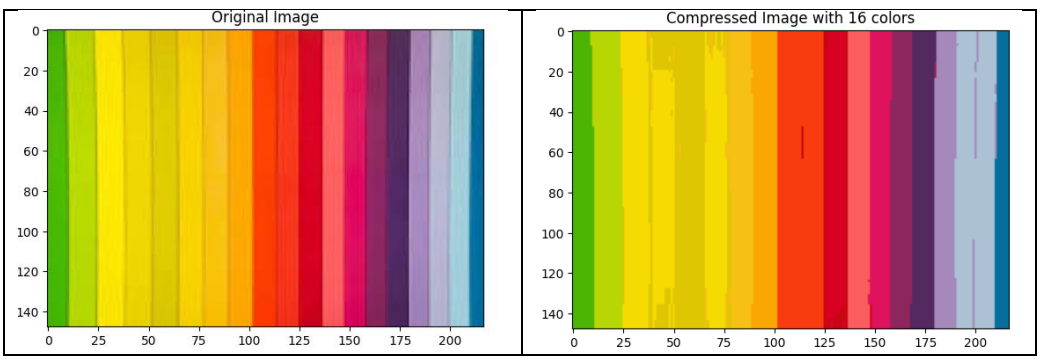
Advantages

- Simple and effective for colour image compression.
- Reduces memory/storage usage.

Limitations

- Image quality reduces as K decreases.
- Computationally expensive for large images.

```python
# pip install scikit-image
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from skimage import io

#Read image
image = io.imread('colorful.jpg')
plt.imshow(image)
plt.title("Original Image")
plt.show()

#Reshape image data
pixels = image.reshape(-1, 3) #converts image pixels into 2D array, one pixel per row, 3 columns for RGB.

#Apply K-Means for color quantization
k = 16   # reduce to 16 colors
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(pixels)
new_colors = kmeans.cluster_centers_[kmeans.labels_] #cluster_centers_, gives representative colors.
                                                     #labels_ → tells which cluster each pixel belongs to.

#Recreate compressed image
compressed = new_colors.reshape(image.shape).astype('uint8') #unsigned 8-bit integers (0–255), is the standard data type for image pixels.

#Display compressed image
plt.imshow(compressed)
plt.title(f"Compressed Image with {k} colors")
plt.show()
```



Mean Shift Clustering Model

- Mean Shift Clustering is a non-parametric clustering technique.
- It means we don't have to specify the number of clusters (K) in advance (unlike K-Means).
- So, instead of assuming a fixed number of clusters, it automatically discovers clusters based on the data.
- Initially, each point is considered as a potential cluster center.
- Then it takes a circular (or spherical) window around the point with a certain bandwidth (radius).
- It computes the mean of all points within that window.
- Shift the window center to the new mean.
- Merge points that converge to the same location and form the cluster.
- Bandwidth is a radius of the neighborhood window.
    - Smaller the bandwidth, forms more clusters.
    - Larger the bandwidth, forms fewer clusters.

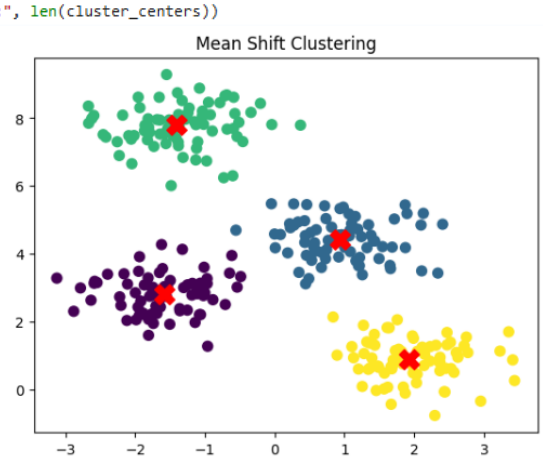
Advantages

- No need to specify number of clusters (unlike K-Means).
- We can find clusters of any shape (not just spherical).
- Automatically ignores noise/outliers.

Limitations

- Computationally expensive for large datasets (since it considers all points).
- Sensitive to the choice of bandwidth.
- May merge close clusters if bandwidth is too large.

```python
# Mean Shift Clustering Model
from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs                    # To generate the data
import matplotlib.pyplot as plt

#Create the data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.6, random_state=0)# make_blobs creates random data points grouped around a few centers
                                                                            # n_samples = 300, total number of records
                                                                            #centers = 4 means number of true clusters in generated data
                                                                            #cluster_std = 0.6 means standard deviation (spread) of each cluster
                                                                            # random_state = 0 means ensures reproducibility
# Estimate a suitable bandwidth automatically
bandwidth = estimate_bandwidth(X, quantile=0.2, n_samples=300) # Bandwidth means radius of the window around each point
                                                               # quantile=0.2 means the how wide is the window

# Create the MeanShift object
ms = MeanShift(bandwidth=bandwidth, bin_seeding=True) # bin_seeding=True means speeds up computation by using initial bin centers

# Fit the model to the data
ms.fit(X)

#Visualize the results
# Plot the original data points, colored by cluster label
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50) #X[:, 0] is the first feature (x-axis), X[:, 1] is the second (y-axis)
                                                              #c=y_kmeans as cluster  size is 4, colors each point according to its
                                                                  #predicted cluster
                                                              #s=50 is the size of the marker
                                                              #cmap='viridis' maps those numeric values to distinct colors using the
                                                                  #viridis color palette

# Overlay the cluster centers on the same plot
# Red 'X' markers represent the centers
plt.scatter(cluster_centers[:, 0], cluster_centers[:, 1],
            c='red', s=200, marker='X') #centers[:,0] and centers[:,1] are centroid coordinates
                                        #c='red' colors the centroid markers red.
                                        #s=200 makes them much larger than data points so they stand out
                                        #marker='X' uses an X symbol to differentiate centroids from data points

# Display the plot
plt.title("Mean Shift Clustering")
plt.show()

# Print number of clusters found
print("Number of clusters found:", len(cluster_centers))
```




```
Number of clusters found: 4
```

Agglomerative Clustering

- It is a tree-based clustering method where the observations are divided into a tree-like structure using distance as a measure.
- This clustering technique uses distance as a measure of similarity and hence observations which have smaller distance are considered as similar and vice-versa.
- Each data point starts as its own cluster.
- Then, pairs of clusters are merged step by step based on their similarity (distance).
- This merging continues until all points belong to one cluster or a stopping criterion (like number of clusters) is reached.
- It forms a hierarchy (tree) called a dendrogram, showing how clusters were merged at each step.

How does it work?

Each data point starts as its own cluster.
The two **closest clusters** are merged.
The process repeats until the desired **number of clusters** is reached.

Advantages

Easy to interpret the results in the dendrogram.

Limitations

It is slower than k-means clustering.
It is affected by noise and outliers in the data.
It cannot be used for large datasets and is not scalable.
It is difficult to handle different sized clusters.

Association Rule mining

- Association Rule Mining (ARM) is a data mining technique used to discover relationships, correlations, or patterns among items in large datasets.
- It finds rules like, "If a customer buys *X*, they are likely to buy *Y* as well."
- Each rule has measurable strengths using metrics like **support**, **confidence**, and **lift**.

Market Basket Analysis

- A data mining technique that is used to uncover purchase patterns in any retail setting is known as Market Basket Analysis.
- It will be helpful to identify associations between products frequently purchased together in transactions to understand customer behavior and improve sales.
- Basically, market basket analysis in data mining involves analysing the combinations of products that are bought together.
- This is a technique that gives the careful study of purchases done by a customer in a supermarket.
- It reveals patterns like "customers who buy bread are also likely to buy butter" and uses this information for strategies such as product placement, cross-selling, and targeted promotions.
- Market basket analysis mainly works with the ASSOCIATION RULE {IF} -> {THEN}.

- Here the 'If A' element is called 'antecedent', and 'Then B' is called as 'Consequent'.

Advantages of Market Basket Analysis

Enhanced Customer Understanding: Retailers can use this information to better understand their customers and make informed decisions.

Improved Inventory Management: Retailers can use this information to make well-informed choices about what products to stock and how to manage their inventory most effectively.

Better Pricing Strategies: A better understanding of the connection between product prices and consumer behavior might help merchants develop better pricing strategies. Using this knowledge, pricing plans that boost sales and profitability can be created.

Sales Growth: Market basket analysis can assist businesses in determining which products are most frequently bought together and where they should be positioned in the store to grow sales. Retailers may boost revenue and enhance customer shopping experiences by improving store layouts and product positioning.

Applications of Market Basket Analysis

Retail
E-commerce
Finance
Telecommunications
Manufacturing

Recommendation Engine

- A recommendation engine is a system designed to suggest items or content to users based on their preferences, behaviors, or past interactions.
- The core goal of a recommendation engine is to personalize the user experience by providing relevant suggestions that enhance user engagement and satisfaction.
- Recommendation engines are available everywhere, influencing our daily interactions with digital platforms, from suggesting movies on streaming services to recommending products on e-commerce websites and even guiding us through potential connections on social media.

Applications of Recommendation Engine

Recommendation engines are widely used across different domains:

- **E-Commerce:** Platforms like Amazon and eBay use recommendation engines to suggest products based on user browsing history, purchase patterns, and similar users' behaviour.
- **Media Streaming:** Services like Netflix and Spotify recommend movies, TV shows, or music tracks based on users' past viewing or listening history and preferences.
- **Social media:** Social networks such as Facebook and Instagram use recommendation algorithms to suggest friends, groups, or posts based on user interactions and interests.
- **News and Content Websites:** Websites like Google News and Medium provide personalized content recommendations based on users' reading habits and preferences.
- **Advertising:** Recommendation engines are employed in targeted advertising to display ads relevant to the user's interests and previous interactions with similar products or services.

Challenges of Recommendation Engine

- **Data Privacy:** Collecting and analysing user data for recommendations must be handled with care to protect user privacy and comply with data protection regulations.
- **Cold Start Problem:** New users or items without sufficient historical data can be challenging for recommendation engines. Various strategies, such as using content-based recommendations or asking users to provide initial preferences, can help address this issue.
- **Scalability:** Handling large volumes of data and providing real-time recommendations require efficient algorithms and robust infrastructure.
- **Bias and Fairness:** Recommendation engines can unintentionally reinforce biases or create echo chambers. Ensuring fairness and diversity in recommendations is important for providing a balanced user experience.

Introduction of Association Rule Learning Algorithms

- Association Rule Learning is a data mining and machine learning technique used to discover hidden relationships and patterns among variables (usually items) in large datasets.
- It helps answer questions like:
  "What items are frequently bought together?"
  "What actions or events tend to occur together?"
- The main goal of ARL is to find interesting associations or correlations among a set of items in transactional or relational data.

Apriori algorithm

- The Apriori Algorithm is one of the most popular algorithms used in Association Rule Learning (ARL) to find frequent item sets and generate association rules from transactional datasets.
- It is the base algorithm upon which other improved algorithms (like FP-Growth and Eclat) are built.

Working of Apriori Algorithm

Step 1: Generate 1-itemsets

Item	Frequency	Support
Bread	4	0.8
Milk	4	0.8
Namkeen	4	0.8
Sweets	3	0.5
Chocolate	1	0.2

Item	Frequency	Support
Soap	2	0.4

If the support is > 0.4, we keep the product:

→ {Bread, Milk, Namkeen, Sweets}

Step 2: Generate 2-itemsets
Combine frequent items into pairs and compute support.

2-itemset	Support
{Bread, Milk}	0.6
{Bread, Namkeen}	0.6
{Milk, Namkeen}	0.6
{Namkeen, Sweets}	0.6
{Milk, Sweets}	0.4
{Bread, Sweets}	0.4

We will continue the combo if the support is ≥ 0.4.

Step 3: Generate 3-itemsets

3-itemset	Support
{Bread, Milk, Namkeen}	0.4
{Milk, Namkeen, Sweets}	0.6
{Bread, Namkeen, Sweets}	0.4

Step 4: Generate Rules
From {Milk, Namkeen, Sweets}, we can form:

- {Milk, Namkeen} → {Sweets}
  - Confidence = 0.6 / 0.5 = 1.2
- {Milk, Sweets} → { Namkeen}
  - Confidence = 0.4 / 0.8 = 0.5

Rule kept: {Milk, Namkeen} → { Sweets} (high confidence)

Measure	Value	Explanation
Support	3/5 = 0.6	Namkeen & Sweets appear together in 3 of 5 transactions
Confidence	3/4 = 0.75	75% of Namkeen buyers also bought Sweets
Lift	0.75 / 0.6 = 1.25	Buying Namkeen increases Sweets purchase by 25% over random chance

FP-Growth Algorithm (Frequent Pattern Growth)
The FP-Growth algorithm is an improved and faster alternative to the Apriori algorithm for mining frequent itemset.
Unlike Apriori, which generates candidate itemset repeatedly, FP-Growth compresses the dataset into a compact structure called an FP-tree (Frequent Pattern Tree) and extracts frequent patterns without generating candidates.
Much faster and memory-efficient than Apriori.

Step-by-Step Working of FP-Growth

We will use the data set below:

Transaction ID	Items Purchased
T1	Bread, Milk
T2	Bread, Namkeen, Sweets, Soap
T3	Milk, Namkeen, Sweets, Coke
T4	Bread, Milk, Namkeen, Sweets
T5	Bread, Milk, Namkeen, Coke

Step 1: Calculate item frequency

Item	Frequency
Bread	4
Milk	4
Namkeen	4
Sweets	3
Coke	2
Soap	1

Now, remove items with frequency < **minimum support threshold**.
Let's assume min_support = 2 → remove "Soap".

Step 2: Sort items in each transaction by descending frequency
Bread (4) → Milk (4) → Namkeen (4) → Sweets (3) → Coke (2)

Transaction ID	Original Items	Sorted Items
T1	Bread, Milk	Bread, Milk
T2	Bread, Sweets, Namkeen	Bread, Namkeen, Sweets
T3	Milk, Namkeen, Sweets, Coke	Milk, Namkeen, Sweets, Coke
T4	Bread, Milk, Namkeen, Sweets	Bread, Milk, Namkeen, Sweets
T5	Bread, Milk, Namkeen, Coke	Bread, Milk, Namkeen, Coke

Step 3: Build the FP-Tree

```
        (root)
          |
        Bread:4
        /     \
   Milk:3    Namkeen:1
    /  \         \
 Namkeen:2 Sweets:1  Sweets:1
    |        \
  Sweets:2     Coke:1
    \
    Coke:1
```

Step 4: Generate Conditional FP-Trees
Item: Coke

Coke appears in two paths:

1. Bread → Milk → Namkeen → Coke
2. Milk → Namkeen → Sweets → Coke

Item: Sweets

Appears in:

1. Bread → Milk → Namkeen → Sweets
2. Bread → Namkeen → Sweets
3. Milk → Namkeen → Sweets → Coke

Item: Namkeen

Appears in multiple paths, often with Bread and Milk.

Step 5: Interpret frequent patterns

"Bread + Milk + Namkeen" appear together in 60% of all transactions 'a very strong combination'

"Namkeen + Sweets" co-occur in 40%

"Coke" mostly appears with "Milk + Namkeen" 'a weaker but notable pattern'

Eclat (Equivalence Class Clustering and bottom-up Lattice Traversal)

Data set

Transaction ID	Items Purchased
T1	Bread, Milk
T2	Bread, Namkeen, Sweets
T3	Milk, Namkeen, Sweets, Coke
T4	Bread, Milk, Namkeen, Sweets
T5	Bread, Milk, Namkeen, Coke

Step 1: Convert to Vertical Data Format

Item	Transaction IDs (TID Set)
Bread	{T1, T2, T4, T5}
Milk	{T1, T3, T4, T5}
Namkeen	{T2, T3, T4, T5}
Sweets	{T2, T3, T4}
Coke	{T3, T5}

Step 2: Compute Support Count

Support count = number of transactions in the TID set.

Example:

- Bread → 4
- Milk → 4
- Namkeen → 4
- Sweets → 3
- Coke → 2

If min_support = 2, all items are frequent.

Step 3: Combine Items using TID set intersections

To find frequent 2-itemsets, intersect the TID sets of two items.

Itemset	Intersection (Common TIDs)	Support
{Bread, Milk}	{T1, T4, T5}	3
{Bread, Namkeen}	{T2, T4, T5}	3
{Bread, Sweets}	{T2, T4}	2
{Bread, Coke}	{T5}	1
{Milk, Namkeen}	{T3, T4, T5}	3
{Milk, Sweets}	{T3, T4}	2
{Namkeen, Sweets}	{T2, T3, T4}	3
{Namkeen, Coke}	{T3, T5}	2

Step 4: Continue Combining (Generate Larger Itemsets)
To find 3-itemsets, again intersect the TID sets of frequent 2-itemsets.
Example:

- {Bread, Milk, Namkeen} = intersection of {T1, T4, T5} ∩ {T2, T4, T5} = {T4, T5} → Support = 2
- {Milk, Namkeen, Sweets} = {T3, T4, T5} ∩ {T2, T3, T4} = {T3, T4} → Support = 2

Step 5: Stop When No Frequent Item sets Can Be Formed
We stop once no further intersections produce sets meeting the minimum support threshold.

Frequent Itemset	Support
{Bread}	4
{Milk}	4
{Namkeen}	4
{Sweets}	3
{Bread, Milk}	3
{Bread, Namkeen}	3
{Milk, Namkeen}	3
{Namkeen, Sweets}	3
{Bread, Milk, Namkeen}	2
{Milk, Namkeen, Sweets}	2